

Solution 16.

* Factorial

```
int factorial (int n)
{
    if (n == 0 || n == 1)
        return 1; // Base Case
    return n * factorial (n-1); // Recursive Call
}
```

Base Case $n=0/n=1$ $T(1)=1$

No of Recursive Call $T(n) = T(n-1) + 1$

$$T(n) = T(n-1) + 1 \quad \text{--- (1)}$$

$$\begin{aligned} T(n-1) &= T(n-1-1) + 1 \\ &= T(n-2) + 1 \quad \text{--- (2)} \end{aligned}$$

put $T(n-1)$ in ~~eqn~~ eqn 1.

$$T(n) = T(n-2) + 2$$

$$T(n) = T(n-3) + 3$$

$$k^{\text{th}} \text{ iteration } T(n) = T(n-k) + k$$

So put $k = n-1$

$$\begin{aligned} T(n) &= T(n - (n-1)) + n-1 \\ &= T(n - n + 1) + n-1 \\ &= T(1) + n-1 \\ &= 1 + n-1 \\ &= n \end{aligned}$$

$$\boxed{T(n) = O(n)}$$

* Binary Search

$$T(n) = \begin{cases} T(n/2) + c & \text{if } n \geq 1 \\ 1 & \text{if } n = 1 \end{cases}$$

$$T(n) = T(n/2) + C \quad (1)$$

$$T(n/2) = T(n/4) + C \quad (2)$$

$$T(n/4) = T(n/8) + C \quad (3)$$

$$T(n) = T(n/4) + C + C$$

$$= T(n/2^2) + 2C$$

$$T(n) = T(n/8) + 2C + C$$

$$= T(n/2^3) + 3C$$

$$T(n) = T(n/2^4) + 4C$$

$\vdots$ k steps

$$T(n) = T(n/2^k) + kC$$

put $2^k = n$

$$T(n) = T(n/n) + kC$$

$$= T(1) + kC$$

$$2^k = n \Rightarrow 1 + \log_2 n \cdot C$$

$$\log_2 2^k = \log_2 n \Rightarrow \underline{O(\log n)}$$

$$k \log_2 2 = \log_2 n$$

$$k = \log_2 n$$